

커맨드 레벨 개별 수집

도구가 실제로 실행하는 명령 전수
상태 수집 · 세션 수집 · 패킷 덤프

도구 없이, 손으로 하나씩 수집할 때 쓴다.

여기 적힌 명령은 전부 `nsx-collector.py` 소스에서 그대로 뽑은 것이다. 도구가 내부에서 이 명령들을 순서대로 돌리고, 출력을 정해진 이름의 파일로 떨군다.

이럴 때 쓴다 — 도구를 올릴 수 없는 장비, 딱 하나만 급히 봐야 할 때, 도구가 만든 결과가 이상해서 직접 대조해 볼 때, 그리고 도구가 무엇을 하는지 이해할 때.

기준	nsx-collector v6.3
검증	NSX 4.2.4 Edge · ESXi 8.0.3 실측
작성일	2026-09-21

목차

A

공통 — 명령을 어디서 치나

3

B

NSX Edge — 상태 수집

5

C

NSX Edge — 세션 수집

7

D

NSX Edge — 패킷 덤프

10

E

ESXi — 상태 수집

13

F

ESXi — 세션 수집 (DFW)

15

G

ESXi — 패킷 덤프

17

H

한 장 요약

20

표기 규칙

표기	뜻
<LIF-UUID>	논리 인터페이스 UUID — 설정의 LIF_LBT1_SVC 등
<SR-UUID>	서비스 라우터 UUID — T1_LB_SR_UUID 등
<filter>	DFW 필터 이름 (nic-12345-eth0-vmware-sfw.2 형태)
<VM>	VM 이름

공통 — 명령을 어디서 치나

NSX Edge — 두 가지 셸이 있다

Edge 에는 **리눅스 셸**과 **NSX CLI** 두 가지가 있고, 명령이 같린다.

셸	어떻게 들어가나	여기서 치는 것
리눅스 셸	<code>ssh root@<edge></code>	<code>tcpdump</code> , <code>ip link</code> , <code>timeout</code>
NSX CLI	<code>ssh admin@<edge></code>	<code>get ...</code> , <code>set capture session ...</code>

root 로 들어가서 NSX CLI 명령을 치는 방법

도구는 root 셸에서 `su admin -c` 로 NSX CLI 명령을 던진다. 이 방식이면 셸을 오가지 않아도 되고, 스크립트로 묶기도 쉽다.

```
su admin -c "get interfaces"
su admin -c "get logical-routers"
```

이 별칭의 Edge 명령은 전부 이 형태로 적는다. `admin` 셸에 직접 들어가 있다면 `su admin -c "..."` 를 빼고 **안쪽 명령만** 치면 된다.

admin 셸에서는 리눅스 명령이 안 된다

`admin` 으로 로그인하면 NSX CLI 셸이라 `tcpdump`, `python3`, `ls` 같은 것이 `% Command not found` 로 거부된다. 패킷 덤프는 반드시 **root 셸**에서 해야 한다.

ESXi — 셸 하나뿐

```
ssh root@<esxi>
```

모든 명령을 여기서 친다. 다만 ESXi 의 `timeout` 은 문법이 일반 리눅스와 다르다 — D · G 장에서 다룬다.

결과를 어디에 남길 것인가

도구는 `run-<호스트>-<날짜>-<시각>/` 아래 `state/` · `session/` · `pcap/` 세 폴더로 나눠 담는다. 손으로 할 때도 같은 구조를 만들어 두면 나중에 분석기로 읽을 수 있다.

```
D=/var/dump/manual-$(hostname)-$(date +%Y%m%d-%H%M%S)      # Edge
D=/vmfs/volumes/<데이터스토어>/manual-$(date +%Y%m%d-%H%M%S)  # ESXi
mkdir -p $D/state $D/session $D/pcap
```

ESXi에서는 /tmp 를 쓰지 말 것

약 250MB 메모리 디스크라 캡처가 금방 멈춘다. 반드시 데이터스토어로.

NSX Edge — 상태 수집

도구의 `stats-run` 이 돌리는 명령 전부. 카운터와 HA 상태.

B-1 → `state/30-edge-interfaces.txt`

인터페이스 목록과 카운터

```
su admin -c "get interfaces"
```

Edge 의 모든 인터페이스와 RX/TX 를 한 번에 본다. 가장 먼저 보는 것.

B-2 → `state/32-edge-dataplane-cpu.txt`

데이터플레인 CPU

```
su admin -c "get dataplane cpu stats"
```

패킷 처리 코어가 포화됐는지. 드롭이 여기서 나는지 확인할 때.

B-3 → `state/33-edge-port-<포트>-stats.txt`

물리(fastpath) 포트별 카운터

```
su admin -c "get interface fp-eth0"
su admin -c "get interface fp-eth1"
```

실측 함정 — 명령 이름이 다르다

NSX 4.2.4 에는 `get interfaces stats` 라는 명령이 아예 없다 (% Command not found). `get interface <이름>` 이 맞고, 이 출력에 이미 RX/TX 패킷·바이트·에러·드롭이 다 들어 있다.

B-4 → `state/34-edge-router-<앞8자>-interface-stats.txt`

라우터별 인터페이스 통계

```
su admin -c "get logical-router <SR-UUID> interfaces stats"
```

설정의 `T0_SR_UUID` · `T1_VPC_SR_UUID` · `T1_LB_SR_UUID` 에 적은 라우터마다 한 번씩. 라우터 하나당 약 2.5초가 걸리니 필요한 것만.

B-5 → state/35-edge-router-<앞8자>-ha-state.txt

라우터 HA 상태 ★

```
su admin -c "get logical-router <SR-UUID> high-availability status"
```

읽는 방향을 반드시 확인할 것

출력 맨 위 **state** 줄이 지금 접속한 이 노드다. 아래쪽 **Peer Routers** 블록은 상대방 이야기다.

이걸 뒤집어 읽어 Standby 에서 캡처하는 사고가 실제로 난다. Standby 에서 뜨면 패킷이 0건이다.

B-6 → state/36-edge-cpu.txt · 37-edge-memory.txt

CPU · 메모리

```
su admin -c "get cpu-stats"
su admin -c "get memory"
```

실측 함정

`get system-stats` 는 NSX 4.2.4 에 없다. 위 두 개로 나눠서 봐야 한다.

B-7. 반복해서 뜨려면

도구는 **INTERVAL** 초마다 **DURATION** 초 동안 반복한다. 손으로 하면 이렇게 된다.

```
D=/var/dump/manual-$(hostname)-$(date +%Y%m%d-%H%M%S); mkdir -p $D/state

END=$(( $(date +%s) + 1200 ))          # DURATION 1200초
while [ $(date +%s) -lt $END ]; do
    TS=$(date +%H%M%S)
    su admin -c "get interfaces"        > $D/state/30-interfaces-$TS.txt
    su admin -c "get dataplane cpu stats" > $D/state/32-dpcpu-$TS.txt
    su admin -c "get interface fp-eth0"  > $D/state/33-fp-eth0-$TS.txt
    su admin -c "get interface fp-eth1"  > $D/state/33-fp-eth1-$TS.txt
    sleep 30                            # INTERVAL 30초
done
```

왜 반복해서 뜨나

카운터는 **누적값**이라 한 번 떠서는 아무 의미가 없다. **두 시점의 차이**가 그 사이에 무슨 일이 있었는지 말해 준다. 최소 2회, 되도록 여러 번 뜬다.

NSX Edge — 세션 수집

도구의 `sess-run` . 방화벽 연결 테이블과 LB 상태.

이 장이 가장 중요하다

방화벽 연결 테이블은 **지나가면 영영 못 얻는다**. 증상이 있는 그 순간에 뜨지 않으면 사후에는 방법이 없다.

C-1 → session/50-edge-fw-conn-count-...txt

방화벽 연결 개수

```
su admin -c "get firewall <LIF-UUID> connection count"
```

먼저 개수만 본다. 테이블이 클 때 전체를 뜨기 전에 규모를 가늠한다.

C-2 → session/51-edge-fw-conn-table-...txt

방화벽 연결 테이블 전체 ★

```
su admin -c "get firewall <LIF-UUID> connection"
```

출력은 이런 모양이다.

```
0x...: 10.1.1.20:54982 -> 10.2.2.30:80 (10.1.1.10:80)
      ↑ 괄호 안에 NAT 매핑
```

괄호 안에 NAT 이전 주소가 같이 나온다. LB 가 경로에 있을 때 "이 연결이 원래 어디로 온 것인지"를 이것 하나로 알 수 있다.

실측 함정 — SR UUID 를 넣으면 안 된다

이 명령은 **논리 인터페이스(LIF) UUID** 를 받는다. 서비스 라우터(SR) UUID 를 넣으면 `% Invalid value for argument <uuid>` 가 나오고, 파일에는 데이터 대신 **에러가 담긴다**.

도구 v5 가 여기서 아무것도 못 모았던 원인이 이것이다. NSX 4.2.4 실측.

설정에 적은 LIF **전부**에 대해 돌린다 — `LIF_LBT1_SVC` , `LIF_VPCT1_UPLINK` , `LIF_T0_UPLINK` , `LIF_VPCT1_DOWNLINK` .

C-3 → session/52-53-edge-lb-...txt

로드밸런서 상태와 통계

```
su admin -c "get load-balancer <LB-UUID> status"
su admin -c "get load-balancer <LB-UUID> stats"
```

C-4 → session/54-edge-lb-virtualservers-...txt

가상 서버 목록

```
su admin -c "get load-balancer <LB-UUID> virtual-servers"
```

C-5 → session/55-edge-lb-pool-<앞8자>-status.txt

풀 상태 — 멤버가 살아 있나

```
su admin -c "get load-balancer <LB-UUID> pool <POOL-UUID> status"
```

백엔드가 전부 DOWN 인지 여기서 확인한다. 풀이 여러 개면 각각.

C-6. UUID 를 모를 때 — 찾는 명령

명령	얻는 것
<code>su admin -c "get logical-routers"</code>	SR UUID 목록
<code>su admin -c "get logical-router <SR-UUID> interfaces"</code>	LIF UUID — <code>Port-type</code> 이 <code>service</code> / <code>uplink</code>
<code>su admin -c "get load-balancers"</code>	LB UUID 목록
<code>su admin -c "get load-balancer <LB-UUID> virtual-servers"</code>	VIP · 포트 · 풀
<code>su admin -c "get load-balancer <LB-UUID> pool <POOL> snat-pools"</code>	SNAT 주소

C-7. 반복해서 뜨려면

```
D=/var/dump/manual-$(hostname)-$(date +%Y%m%d-%H%M%S); mkdir -p $D/session
LIF=<LIF-UUID>

END=$(( $(date +%s) + 1200 ))
while [ $(date +%s) -lt $END ]; do
    TS=$(date +%H%M%S)
    su admin -c "get firewall $LIF connection count" > $D/session/50-conn-count-$TS.txt
    su admin -c "get firewall $LIF connection" > $D/session/51-conn-table-$TS.txt
    sleep 30
done
```

세션 테이블도 여러 번 떠나 한다

연결이 생겼다 사라지는 과정을 봐야 하기 때문이다. 한 장만 있으면 "그때 세션이 있었나 없었나"만 알 수 있고, **언제 사라졌는지는 모른다.**

NSX Edge — 패킷 덤프

도구의 `cap-lbt1` · `cap-vpct1` · `cap-t0` . span 미러를 만들고 tcpdump 로 받는다.

원리 — 인터페이스를 직접 뜯 수 없다

NSX Edge 의 논리 인터페이스는 **tcpdump** 로 직접 지정할 수 없다. 그래서 **span 미러**를 만들어 `span-N` 이라는 가상 인터페이스를 띄우고, 거기에 tcpdump 를 붙인다.

```
<LIF-UUID> —(미러)—> span-0 —> tcpdump
```

3단계로 한다

D-1 · NSX CLI

① span 미러 생성

```
su admin -c "set capture session 0 interface <LIF-UUID> direction dual"
```

`0` 은 세션 번호(0~3). `dual` 은 양방향.

생성되면 `span-0` 인터페이스가 리눅스 쪽에 나타난다. 확인:

```
ip link show span-0
```

최대 10초쯤 기다려야 할 수 있다

도구도 1초 간격으로 10번까지 확인하고, 안 뜨면 **세션을 지우고 실패** 처리한다 (미러를 남기지 않기 위해). 안 뜨면 **LIF UUID** 를 다시 확인한다.

D-2 · 리눅스 셸 (root)

② tcpdump 로 받기 ★

```
timeout -s INT 300 tcpdump -nei span-0 \  
-Z root \  
-C 100 -W 5 \  
-w /var/dump/manual/pcap/10-lbt1svc-$(date +%H%M%S).pcap \  
'<filter>'
```

옵션	왜 필요한가
-Z root	반드시 넣는다. tcpdump 는 기본적으로 권한을 낮추는데, 그러면 링 버퍼의 다음 파일을 못 만든다
timeout -s INT 300	300초 뒤 INT 시그널로 정상 종료. 그래야 마지막 파일이 온전하다
-C 100 -W 5	파일당 100MB, 5개까지 = 최대 500MB. 넘으면 오래된 것부터 덮어쓴다
-nei	n 이름해석 안 함 · e L2 헤더 표시 · i 인터페이스
-s 128	(선택) 패킷당 128바이트만. 헤더만 볼 때 용량 절약

D-3 · NSX CLI

③ span 미러 삭제 — 반드시

```
su admin -c "del capture session 0"
```

남겨두면 안 된다

미러는 계속 패킷을 복사한다. 운영 장비에 남기면 부하가 된다. 도구는 tcpdump 가 어떻게 끝나든 finally 로 반드시 지운다. 손으로 할 때는 잊기 쉬우니 먼저 메모해 둘 것.

남은 미러 확인:

```
su admin -c "get capture session 0"  
ip link show | grep span
```

D-4. 한 번에 붙여 쓰기

```
LIF=<LIF-UUID>
D=/var/dump/manual-$(date +%Y%m%d-%H%M%S); mkdir -p $D/pcap

su admin -c "set capture session 0 interface $LIF direction dual"
sleep 3; ip link show span-0 || echo "span 이 안 났다 - LIF 확인"

timeout -s INT 300 tcpdump -nei span-0 -Z root -C 100 -W 5 \
-w $D/pcap/10-lbt1svc-$(date +%H%M%S).pcap \
'host 10.1.1.10 and tcp port 443'

su admin -c "del capture session 0"      # ← 잊지 말 것
ls -la $D/pcap/
```

D-5. 어느 LIF 를 뜯 것인가

지점	무엇이 보이나	세션 번호 예
LB T1 서비스	VIP 로 들어온 트래픽, LB → 백엔드	0
VPC T1 업링크	NAT 이전 트래픽 — VIP 가 안 보인다	1
T0 업링크	사이트로 들어오는 전부. 매우 시끄럽다	2 , 3

캡처가 0건일 때 — 순서대로 본다

- 이 Edge 가 Standby 인가 — B-5 의 HA 상태 확인. Standby 면 0건이다
- 그 시간 동안 매칭되는 트래픽이 실제로 있었나
- 필터가 이 지점에 안 맞나 — VPC T1 업링크는 NAT 이전이라 VIP 가 절대 안 나타난다. 공인/NAT 주소로 걸거나 필터를 비운다

T0 는 A/A — 모든 Edge 에서 동시에 떼야 한다

T0 는 보통 Active/Active 라 트래픽이 노드마다 나뉜다. 한 노드만 뜨면 절반만 보인다. 그리고 T0 업링크는 보통 2개라 노드당 2개를 떼야 한다.

ESXi — 상태 수집

도구의 `stats-run` (ESXi). 스위치포트와 업링크 카운터.

E-1 → state/40-esxi-switchport-list.txt

스위치포트 목록 ★ 먼저 이것부터

```
net-stats -l
```

출력이 이렇다.

PortNum	Type	SubType	SwitchName	MACAddress	ClientName
67108942	5	9	DvsPortset-0	00:50:56:xx:xx:xx	web01.eth0
67108988	5	9	DvsPortset-0	00:50:56:yy:yy:yy	web02.eth0

맨 왼쪽 PortNum 이 뒤에 계속 필요하다. 맨 오른쪽에서 VM 이름을 찾아 그 줄의 PortNum 을 쓴다.

E-2 → state/41-esxi-uplink-<nic>-stats.txt

물리 NIC 카운터

```
esxcli network nic stats get -n vmnic0
esxcli network nic stats get -n vmnic1
```

설정의 UPLINK_NICS 에 적은 NIC 마다. 물리 구간 드롭·에러를 본다.

E-3 → state/42-esxi-vmport-<VM>-<port>-stats.txt

VM 포트별 상세 카운터

```
net-stats -A -t vW -p 67108942 -i 1 -n 1
```

옵션	뜻
-p	E-1 에서 찾은 PortNum
-A	모든 통계
-t vW	vNIC · 월드(world) 단위
-i 1 -n 1	1초 간격으로 1회 — 안 주면 계속 돈다

E-4. 반복해서 뜨려면

```
D=/vmfs/volumes/<데이터스토어>/manual-$(date +%Y%m%d-%H%M%S); mkdir -p $D/state
PORT=67108942

END=$(( $(date +%s) + 1200 ))
while [ $(date +%s) -lt $END ]; do
    TS=$(date +%H%M%S)
    net-stats -l >> $D/state/40-switchports.txt
    esxcli network nic stats get -n vmnic0 > $D/state/41-vmnic0-$TS.txt
    net-stats -A -t vW -p $PORT -i 1 -n 1 > $D/state/42-port$PORT-$TS.txt
    sleep 30
done
```

ESXi — 세션 수집 (DFW)

도구의 `dfw-run` . 플로우 테이블 · 적용된 룰 · pass/drop 카운터.

F-0. 먼저 DFW 필터 이름을 찾는다 — 모든 명령의 출발점

```
summarize-dvfilter
```

출력에서 VM 을 찾는다.

```
world 2098765 vmm0:web01 vcUuid:'...'
port 67108942 web01.eth0
vNic slot 2
  name: nic-2098765-eth0-vmware-sfw.2    # ← 이것이 <filter>
  agentName: vmware-sfw
```

읽는 법

- `world ... vmm0:<VM이름>` — 이 줄로 VM 을 찾는다
- `name: ...vmware-sfw.N` — 이것이 DFW 필터 이름
- vNIC 이 여러 개면 필터도 여러 개다. 전부 떼야 한다

F-1 → session/60-dfw-filter-list.txt

필터 목록 전체를 보관

```
summarize-dvfilter > $D/session/60-dfw-filter-list.txt
```

나중에 "그때 이 VM 이 어느 포트·어느 필터였나"를 대조하려면 원본을 남겨야 한다.

F-2 → session/61-dfw-flows-<VM>-<nic>-...txt

DFW 플로우 테이블 ★

```
vsipioctl getflows -f <filter>
```

이 vNIC 을 지나는 연결이 지금 몇 개고 어떤 것인지. Edge 의 `get firewall ... connection` 에 해당하는 ESXi 쪽 명령이다.

사후에 얻을 수 없다. 증상 진행 중에 떼야 한다.

F-3 → session/62-dfw-rules-<VM>-<nic>.txt

이 vNIC 에 실제로 적용된 룰

```
vsipioctl getrules -f <filter>
```

UI 에 보이는 정책이 아니라 커널에 실제로 내려간 룰이다. "정책은 만들었는데 왜 안 먹나"를 볼 때 여기서 확인한다.

F-4 → session/63-dfw-passdrop-<VM>-<nic>.txt

pass / drop 카운터 ★

```
vsipioctl getfilterstat -f <filter>
```

출력에서 `v4 pass` · `v4 drop` 줄을 본다.

가장 빠른 판정 도구

두 시점에 떠서 `drop` 이 늘었는지만 보면 "DFW 가 막고 있나"가 바로 나온다. 늘지 않았으면 DFW 가 아니라 다른 구간을 봐야 한다.

F-5. 한 번에 붙여 쓰기

```
D=/vmfs/volumes/<데이터스토어>/manual-$(date +%Y%m%d-%H%M%S); mkdir -p $D/session
F=nic-2098765-eth0-vmware-sfw.2 # F-0 에서 찾은 필터
```

```
summarize-dvfilter > $D/session/60-filter-list.txt
```

```
END=$(( $(date +%s) + 1200 ))
while [ $(date +%s) -lt $END ]; do
    TS=$(date +%H%M%S)
    vsipioctl getflows -f $F > $D/session/61-flows-$TS.txt
    vsipioctl getrules -f $F > $D/session/62-rules-$TS.txt
    vsipioctl getfilterstat -f $F > $D/session/63-passdrop-$TS.txt
    sleep 30
done
```

ESXi — 패킷 덤프

도구의 `cap-pre` · `cap-post` . DFW 적용 전/후를 같은 vNIC 에서 뜬다.

원리 — 전과 후를 같이 떼서 대조한다

vNIC → [pre] → DFW 룰 평가 → [post] → 스위치

pre 에는 있는데 post 에 없다 = **DFW 가 막았다**

이것이 ESXi 캡처의 핵심이다. **추정이 아니라 증거**가 된다.

먼저 알아야 할 것 — pktcap-uw 에는 BPF 필터가 없다

tcpdump 처럼 'host A and port B' 같은 표현식을 못 쓴다. `--ip` , `--tcpport` 같은 개별 옵션만 있고 `or` 조합이 안 된다.

그래서 방법이 둘로 갈린다 — **G-1**(옵션만) 또는 **G-2**(tcpdump-uw 로 파이프, 권장).

G-0. ESXi 의 timeout 문법부터 확인

ESXi 의 `timeout` 은 버전에 따라 문법이 다르다. **둘 중 되는 것을 쓴다.**

```
timeout -t 1 -s INT true    && echo "형식 A: timeout -t <초> -s INT"
timeout -s INT 1 true      && echo "형식 B: timeout -s INT <초>"
```

pktcap-uw 의 -G 는 캡처 시간이 아니다

`-G` 는 **파일 회전 주기**다. 캡처를 시간으로 끝내려면 반드시 **바깥에서** `timeout` 으로 감싸야 한다.

G-1 · 옵션만 쓰는 방식

단순한 조건일 때

```
echo Y | timeout -t 300 -s INT \  
  pktcap-uw --dvfilter <filter> --stage pre \  
  --ip 10.1.1.10 --tcpport 80 \  
  -C 100 -W 5 \  
  -o $D/pcap/20-dfw-pre-web01-eth0.pcap
```

옵션	뜻
<code>--dvfilter</code>	F-0 에서 찾은 DFW 필터 이름
<code>--stage pre post</code>	DFW 적용 전 / 후. 두 번 떼서 대조한다
<code>--ip</code> · <code>--tcpport</code>	필터 옵션. or 조합 불가
<code>-C 100 -W 5</code>	파일당 MB · 개수
<code>echo Y </code>	확인 프롬프트를 자동으로 넘긴다

조건이 여러 개면 파일이 폭증한다

`or` 가 없으니 **조합마다 별도 캡처**를 돌려야 한다. IP 2개 × 포트 2개면 vNIC 당 4개. vNIC 2개면 8개 파일이 된다.

G-2 · tcpdump-uw 로 파이프 권장

복잡한 필터를 쓰고 파일을 하나로

```
echo Y | timeout -t 300 -s INT \  
  pktcap-uw --dvfilter <filter> --stage pre -o - 2>/dev/null \  
  | tcpdump-uw -r - -w $D/pcap/20-dfw-pre-web01-eth0.pcap \  
  -C 100 -W 5 'host 10.1.1.10 and tcp port 80'
```

이 방식의 장점

- **tcpdump 표현식을 그대로 쓴다** — `or` , `not` , 괄호 전부 가능
- **vNIC·단계당 파일 하나** — 파일 폭증이 없다
- 도구가 **FILTER** 설정이 있을 때 쓰는 방식이 이것이다

`pktcap-uw -o -` 로 표준출력에 흘리고, `tcpdump-uw -r -` 로 받아서 필터링·저장한다.

G-3. pre 와 post 를 같이 — 실제로 쓰는 형태

```
D=/vmfs/volumes/<데이터스토어>/manual-$(date +%Y%m%d-%H%M%S); mkdir -p $D/pcap
F=nic-2098765-eth0-vmware-sfw.2
EXPR='host 10.1.1.10 and tcp port 80'

for S in pre post; do
  N=20; [ $S = post ] && N=21
  echo Y | timeout -t 300 -s INT \
    pktcap-uw --dvfilter $F --stage $S -o - 2>/dev/null \
    | tcpdump-uw -r - -w $D/pcap/$N-dfw-$S-web01-eth0.pcap -C 100 -W 5 "$EXPR" &
done
wait
ls -la $D/pcap/
```

반드시 동시에 떼야 한다

pre 와 post 를 **따로 뜨면 대조가 안 된다**. 같은 패킷을 봐야 하므로 & 로 동시에 띄우고 wait 로 기다린다.

G-4. 캡처가 안 될 때

증상	확인
바로 끝나고 파일이 0바이트	2>/dev/null 을 빼고 다시 돌려 에러를 본다. 필터 이름 오타가 가장 흔하다
프롬프트에서 멈춤	echo Y 를 빠뜨렸다
끝나지 않는다	timeout 문법이 이 장비와 안 맞는다 → G-0
패킷이 0건	필터가 안 맞거나, 그 시간에 트래픽이 없었다
디스크가 찼다	/tmp 에 쓰고 있다 → 데이터스토어로

한 장 요약

출력해서 옆에 두고 쓰는 용도.

NSX Edge

구분	명령
상태	su admin -c "get interfaces"
	su admin -c "get dataplane cpu stats"
	su admin -c "get interface fp-eth0" (fp-eth1 도)
	su admin -c "get logical-router <SR> interfaces stats"
	su admin -c "get logical-router <SR> high-availability status" ★
	su admin -c "get cpu-stats" · su admin -c "get memory"
세션	su admin -c "get firewall <LIF> connection count"
	su admin -c "get firewall <LIF> connection" ★
	su admin -c "get load-balancer <LB> status" · stats
	su admin -c "get load-balancer <LB> virtual-servers"
	su admin -c "get load-balancer <LB> pool <P00L> status"
패킷	su admin -c "set capture session 0 interface <LIF> direction dual"
	timeout -s INT 300 tcpdump -nei span-0 -Z root -C 100 -W 5 -w <파일> '<식>'
	su admin -c "del capture session 0" ← 반드시

구분	명령
상태	<code>net-stats -l ★</code> (PortNum 을 여기서 찾는다)
	<code>esxcli network nic stats get -n vmnic0</code>
	<code>net-stats -A -t vW -p <PortNum> -i 1 -n 1</code>
세션	<code>summarize-dvfilter ★</code> (필터 이름을 여기서 찾는다)
	<code>vsipioctl getflows -f <filter> ★</code>
	<code>vsipioctl getrules -f <filter></code>
	<code>vsipioctl getfilterstat -f <filter> ★</code>
패킷	<code>echo Y timeout -t 300 -s INT pktcap-uw --dvfilter <f> --stage pre --ip <IP> -o <파일></code>
	<code>echo Y timeout -t 300 -s INT pktcap-uw --dvfilter <f> --stage pre -o - tcpdump-uw -r - -w <파일> '<식>'</code>

실측으로 확인된 함정 6가지

함정	내용
<code>get interfaces stats</code>	그런 명령은 없다(NSX 4.2.4). <code>get interface <이름></code> 을 쓴다
<code>get system-stats</code>	없다. <code>get cpu-stats</code> + <code>get memory</code> 로 나눈다
<code>get firewall <SR> connection</code>	SR 이 아니라 LIF UUID 를 넣어야 한다. SR 을 넣으면 에러가 파일에 담긴다
HA 상태의 <code>Peer Routers</code>	맨 위 <code>state</code> 가 이 노드, <code>Peer</code> 블록은 상대방
<code>tcpdump -Z root</code>	빠뜨리면 링 버퍼 두 번째 파일부터 못 쓴다
<code>pktcap-uw -G</code>	캡처 시간이 아니라 파일 회전 주기. 종료는 <code>timeout</code> 으로

손으로 할 때 가장 많이 잊는 것

- 1. span 미러를 안 지운다 — 운영 장비에 계속 부하가 남는다
- 2. 한 번만 뜯다 — 카운터·세션은 두 시점 이상 있어야 의미가 있다
- 3. Standby Edge 에서 뜯다 — 0건이 나온다
- 4. ESXi `/tmp` 에 쓴다 — 250MB 램디스크라 금방 찬다

